



Magíster en Estadística, PUCV

Tree-Based Methods and Ensembles

Alejandro Veloz

Overview

1. Why combining models can beat choosing one.
2. Decision trees: recursive, greedy partitioning.
3. Impurity, information gain, and regression trees.
4. Overfitting and cost-complexity pruning.
5. Bagging and random forests: averaging decorrelated trees.
6. Boosting: AdaBoost and gradient boosting.
7. Mixtures of experts, and tree ensembles in practice.

Why combine models

Ensembles

Suppose we have several imperfect predictors

$$f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x}).$$

An ensemble combines them:

$$F(\mathbf{x}) = \sum_{m=1}^M \alpha_m f_m(\mathbf{x}).$$

Ensembles

Suppose we have several imperfect predictors

$$f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x}).$$

An ensemble combines them:

$$F(\mathbf{x}) = \sum_{m=1}^M \alpha_m f_m(\mathbf{x}).$$

The goal is not to make every model perfect.

The goal is to make their mistakes cancel when combined.

Ensemble intuition

For classification:

$$F(\mathbf{x}) = \text{mode}\{f_1(\mathbf{x}), \dots, f_M(\mathbf{x})\}.$$

For regression:

$$F(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M f_m(\mathbf{x}).$$

Averaging reduces variance.

Voting can cancel individual mistakes.

Three ways to combine models:

- **Average** numeric predictions.
- **Vote** for class labels.
- **Learn weights** when some models are more reliable.

The gain depends on both:

- Individual accuracy.
- Diversity of errors.

Diversity and the variance of an average

If the M models have variance σ^2 and pairwise correlation ρ :

$$\text{Var}(\bar{f}) = \rho \sigma^2 + \frac{1 - \rho}{M} \sigma^2.$$

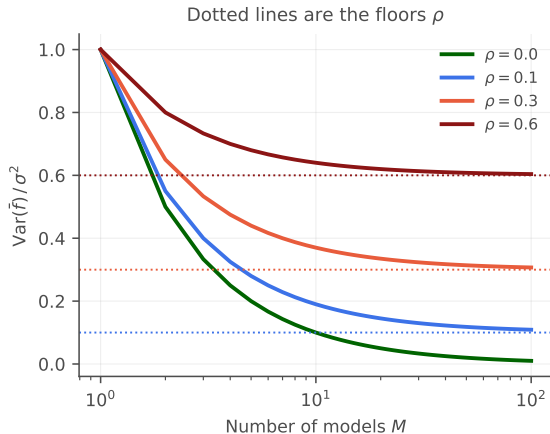
Diversity and the variance of an average

If the M models have variance σ^2 and pairwise correlation ρ :

$$\text{Var}(\bar{f}) = \rho \sigma^2 + \frac{1 - \rho}{M} \sigma^2.$$

As $M \rightarrow \infty$ the second term vanishes and the first stays.

Adding models cannot beat the correlation floor $\rho \sigma^2$.



Negative correlation learning

The correlation can also be penalized explicitly. Each model minimizes

$$\mathcal{L}_m = \sum_{i=1}^N (y_i - f_m(\mathbf{x}_i))^2 + \lambda \sum_{i=1}^N (f_m(\mathbf{x}_i) - \bar{f}(\mathbf{x}_i)) \times \sum_{\ell \neq m} (f_\ell(\mathbf{x}_i) - \bar{f}(\mathbf{x}_i)),$$

where $\bar{f}(\mathbf{x}_i) = \frac{1}{M} \sum_{m=1}^M f_m(\mathbf{x}_i)$.

The penalty discourages learners from making the same deviations.

Liu & Yao (1999). Trained jointly, typically with neural networks.

Three ensemble families

Method	How diversity appears	Combination
Bagging	different bootstrap datasets	average / majority vote
Random forest	bootstrap + random features	average / majority vote
Boosting	sequential focus on mistakes	weighted sum
Mixture of experts	different regions or regimes	learned gate

Trees are a natural base learner: high variance, fast, and interpretable.

Decision trees

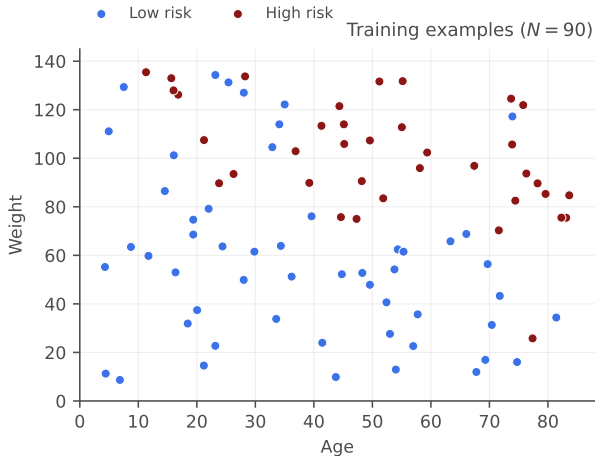
Example

We want to predict patient risk from:

- Age.
- Weight.
- A binary label.

High risk and low risk form regions in feature space.

A tree approximates these regions with axis-aligned rules.



Decision rules

A tree is a collection of rules:

IF Weight > 70 AND Age > 40 THEN High risk.

Each root-to-leaf path is one rule.

Learning a tree means learning the rules from data:

which variable to ask about, which threshold, and in which order.

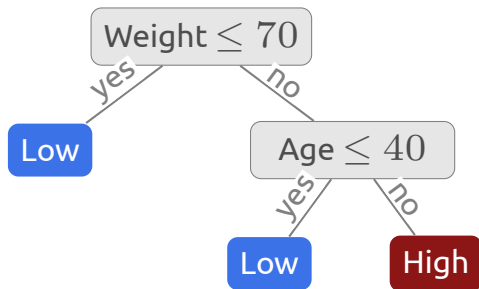
What is a decision tree?

A decision tree is a hierarchical predictive model.

- Internal nodes ask questions.
- Branches are answers.
- Leaves make predictions.

For tabular data, the usual question is

$$x_j \leq t?$$



Recursive partitioning

Tree learning is **top-down recursive partitioning**.

1. Choose the best split at the current node.
2. Divide the data into two child nodes.
3. Repeat the same procedure in each child.
4. Stop when a criterion is met.

The algorithm is greedy: it optimizes the next split, not the whole tree.

Step 1: evaluate candidate splits

```
def best_split(X_node, y_node, min_leaf=5):
    best = None
    for j in range(X_node.shape[1]):          # every feature
        for t in candidate_thresholds(X_node[:, j]):  # every midpoint
            left = X_node[:, j] <= t
            if left.sum() < min_leaf or (~left).sum() < min_leaf:
                continue
            gain = split_gain(y_node, y_node[left], y_node[~left])
            if best is None or gain > best["gain"]:
                best = {"feature": j, "threshold": t, "gain": gain}
    return best
```

Candidate thresholds are the midpoints between consecutive distinct values.

Implementation

Real implementations avoid the naive cost of trying every split from scratch.

Common optimizations:

- Sort each feature once, then scan thresholds in order.
- Update class counts incrementally while moving the threshold.
- Discard invalid splits early (`min_samples_leaf`, missing values).
- Subsample features, and parallelize across nodes or trees.
- Use histograms of binned features in large-scale gradient boosting.

The algorithm is simple; production code is mostly efficient search.

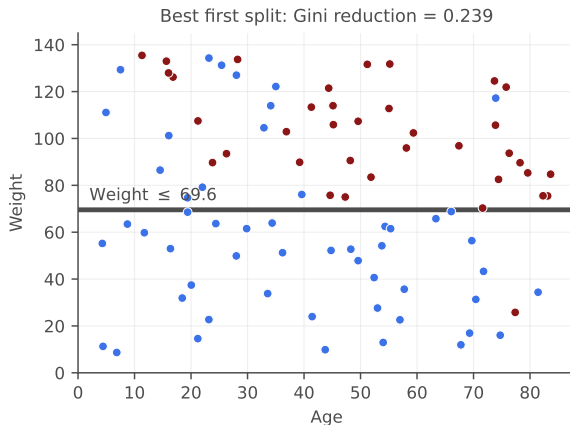
The first split

At the root node, the algorithm asks:

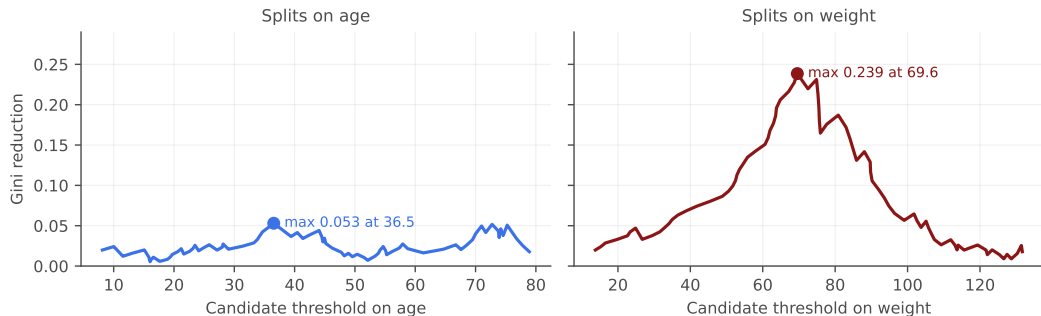
Which feature and threshold most reduce impurity?

Here the best first split is a horizontal cut on weight.

It creates two children with different class mixtures.



Searching over thresholds



The winner is the largest value over *both* panels.

Impurity

A node is pure when almost all its examples share the same label.

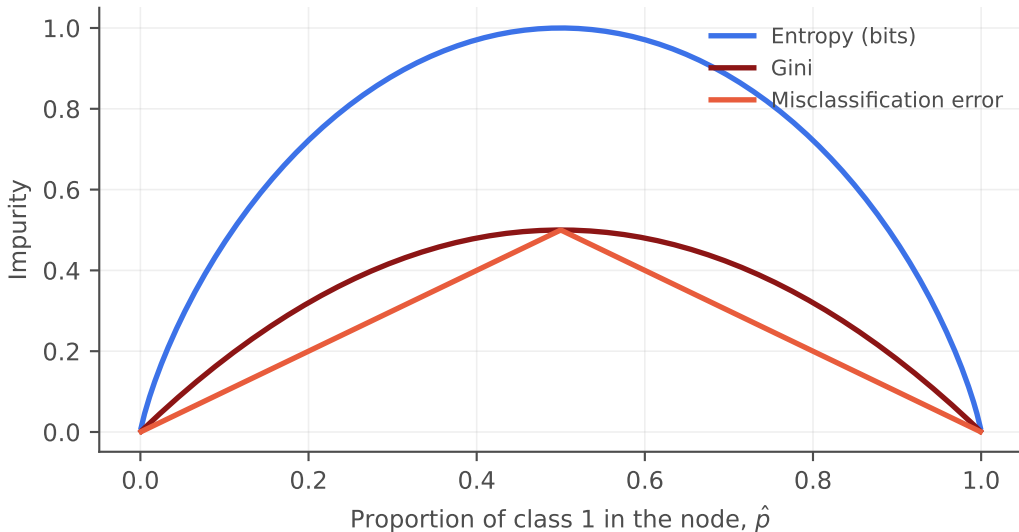
For class proportions $\hat{p}_1, \dots, \hat{p}_K$ in the node:

$$\text{Entropy} = - \sum_{k=1}^K \hat{p}_k \log_2 \hat{p}_k, \quad \text{Gini} = 1 - \sum_{k=1}^K \hat{p}_k^2 = \sum_{k=1}^K \hat{p}_k (1 - \hat{p}_k).$$

Both are zero for pure nodes and largest for evenly mixed nodes.

Impurity curves

Impurity is highest when the classes are mixed



Information gain

For a candidate split of parent P into children L and R :

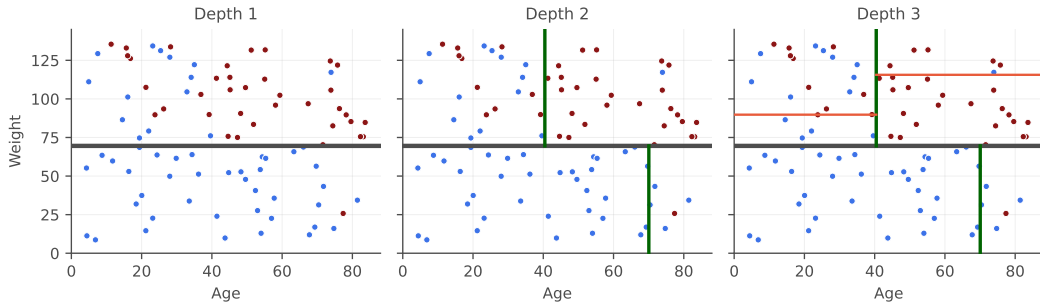
$$\Delta Q = Q(P) - \left(\frac{N_L}{N_P} Q(L) + \frac{N_R}{N_P} Q(R) \right),$$

where $Q(\cdot)$ is an impurity measure and $N_P = N_L + N_R$.

Good splits produce purer child nodes.

With entropy, ΔQ is the *information gain*: the mutual information between the split and the label in the node.

Recurring on each child



The same search, applied inside each region. Each new line stays inside its parent's region.

Stopping

Common stopping criteria:

- Maximum depth.
- Minimum number of samples per leaf.
- Minimum impurity decrease.
- Pure node, or no valid split left.

Stopping controls the bias-variance tradeoff:

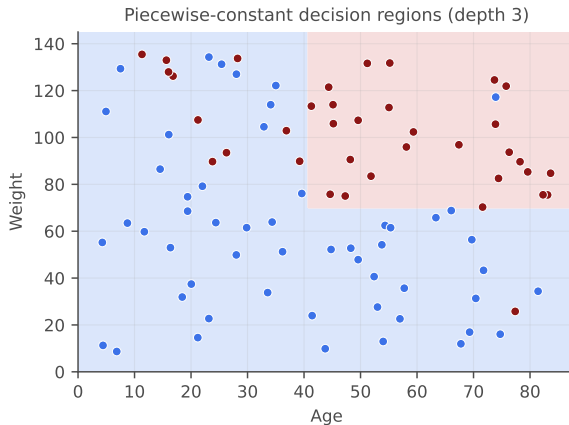
shallow tree $\Rightarrow$ high bias, deep tree $\Rightarrow$ high variance.

Final tree regions

The tree predicts a constant class inside each leaf.

This gives:

- Interpretable rules.
- Non-linear decision boundaries.
- Abrupt changes at split thresholds.



Trees are flexible but unstable.

Classification vs regression trees

Task	Leaf prediction	Split criterion
Classification	majority class / class proportions	Gini or entropy reduction
Regression	mean response in the region	squared-error reduction

Both use the same recursive partitioning.

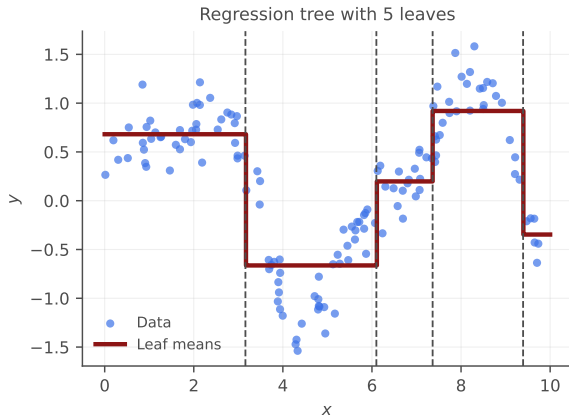
Regression trees

For a leaf region R_m the prediction is

$$\hat{c}_m = \frac{1}{N_m} \sum_{i: \mathbf{x}_i \in R_m} y_i,$$

and the split minimizes

$$\sum_{i: \mathbf{x}_i \in R_L} (y_i - \hat{c}_L)^2 + \sum_{i: \mathbf{x}_i \in R_R} (y_i - \hat{c}_R)^2.$$



A tree is a linear model in a learned basis

A tree with J leaves is

$$T(\mathbf{x}) = \sum_{m=1}^J c_m \phi_m(\mathbf{x}), \quad \phi_m(\mathbf{x}) = 1\{\mathbf{x} \in R_m\}.$$

A tree is a linear model in a learned basis

A tree with J leaves is

$$T(\mathbf{x}) = \sum_{m=1}^J c_m \phi_m(\mathbf{x}), \quad \phi_m(\mathbf{x}) = 1\{\mathbf{x} \in R_m\}.$$

Given the regions, the c_m are least squares in the basis ϕ_m , and the solution is the leaf mean $\hat{c}_m$ (the basis is orthogonal).

A tree is a linear model in a learned basis

A tree with J leaves is

$$T(\mathbf{x}) = \sum_{m=1}^J c_m \phi_m(\mathbf{x}), \quad \phi_m(\mathbf{x}) = 1\{\mathbf{x} \in R_m\}.$$

Given the regions, the c_m are least squares in the basis ϕ_m , and the solution is the leaf mean $\hat{c}_m$ (the basis is orthogonal).

Lecture 1 fixed the basis $\phi_j(\mathbf{x})$ and estimated the weights.

A tree estimates the basis itself, greedily.

Overfitting and pruning

Deep trees can memorize training noise.

Two common controls:

- **Pre-pruning**: stop early using depth, leaf size, or impurity decrease.
- **Post-pruning**: grow a large tree, then remove branches using validation.

The best tree is rarely the largest tree.

Cost-complexity pruning

Grow a large tree T_0 , then for each $\alpha \geq 0$ find the subtree minimizing

$$C_\alpha(T) = \sum_{m=1}^{|T|} N_m Q_m(T) + \alpha |T|,$$

where $|T|$ is the number of leaves.

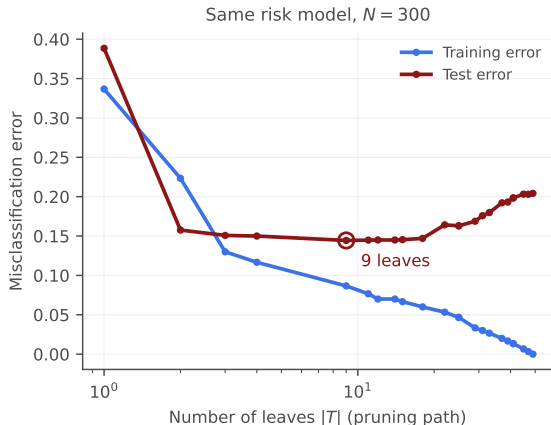
Cost-complexity pruning

Grow a large tree T_0 , then for each $\alpha \geq 0$ find the subtree minimizing

$$C_\alpha(T) = \sum_{m=1}^{|T|} N_m Q_m(T) + \alpha |T|,$$

where $|T|$ is the number of leaves.

Fit + penalty $\times$ complexity, as in ridge and lasso. Choose α by cross-validation.



Why a single tree is not enough

Decision trees have high variance:

- Small changes in the data can change the root split.
- A change near the root propagates to every descendant.
- Deep trees fit local noise.

This makes trees excellent candidates for averaging.

Bagging and random forests

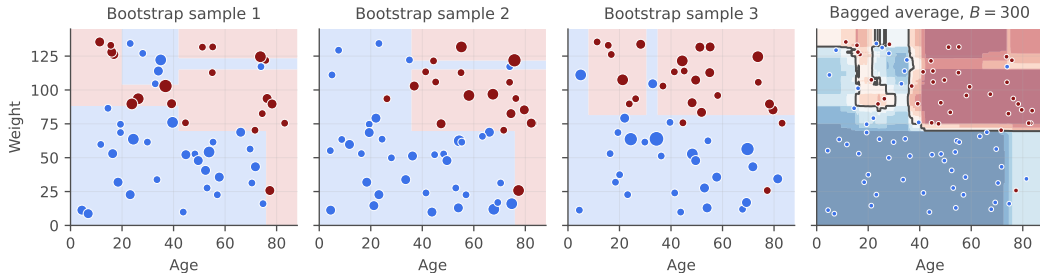
Bagging

Bagging = **b**ootstrap **a**ggregating (Breiman, 1996).

1. Draw B bootstrap samples $\mathcal{D}^{*1}, \dots, \mathcal{D}^{*B}$ from the training set.
2. Grow one deep tree T_b on each sample.
3. Aggregate by averaging or voting:

$$\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}).$$

Bootstrap samples give different trees



Point size is the number of copies in the sample. Missing points are out of bag.

Random forest

Random forest (Breiman, 2001) adds a second source of diversity:

bootstrap samples + random feature subsets at each split.

At each node, only $m < p$ randomly chosen features are candidates.

Random forest

Random forest (Breiman, 2001) adds a second source of diversity:

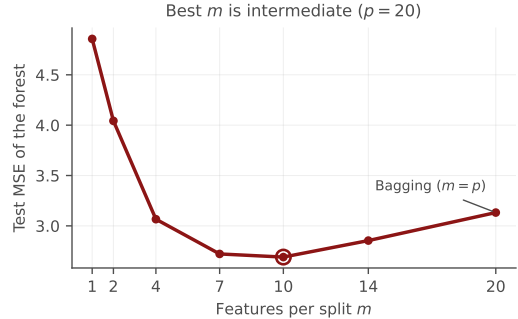
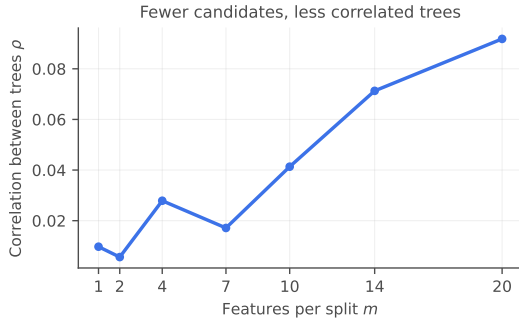
bootstrap samples + random feature subsets at each split.

At each node, only $m < p$ randomly chosen features are candidates.

This decorrelates the trees and lowers the floor $\rho \sigma^2$.

Defaults: $m = \lfloor \sqrt{p} \rfloor$ for classification, $m = \lfloor p/3 \rfloor$ for regression.

Decorrelation, measured



Random forest algorithm

For $b = 1, \dots, B$:

1. Draw a bootstrap sample $\mathcal{D}^{*b}$ of size N .
2. Grow a deep tree T_b on $\mathcal{D}^{*b}$: at each node, sample m of the p features and split on the best of them.

Prediction:

$$\hat{y} = \text{mode}\{T_1(\mathbf{x}), \dots, T_B(\mathbf{x})\} \quad \text{or} \quad \hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}).$$

More trees never cause overfitting; they only stop helping.

Out-of-bag error and feature importance

Each observation is left out of about $e^{-1} \approx 37\%$ of the bootstrap samples.

Predict observation i using only the trees that did **not** see it:

$$\hat{f}_{\text{oob}}(\mathbf{x}_i) = \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} T_b(\mathbf{x}_i), \quad \mathcal{B}_i = \{b : i \notin \mathcal{D}^{*b}\}.$$

This gives, without a separate validation set:

- An estimate of generalization error.
- Permutation feature importance.
- A check of whether more trees still help.

Boosting

Boosting

Boosting builds an additive model sequentially:

$$F_M(\mathbf{x}) = \sum_{m=1}^M \alpha_m h_m(\mathbf{x}).$$

Each new weak learner h_m focuses on what the current ensemble F_{m-1} gets wrong.

Boosting

Boosting builds an additive model sequentially:

$$F_M(\mathbf{x}) = \sum_{m=1}^M \alpha_m h_m(\mathbf{x}).$$

Each new weak learner h_m focuses on what the current ensemble F_{m-1} gets wrong.

Bagging reduces variance by averaging in parallel.

Boosting reduces bias by correcting in sequence.

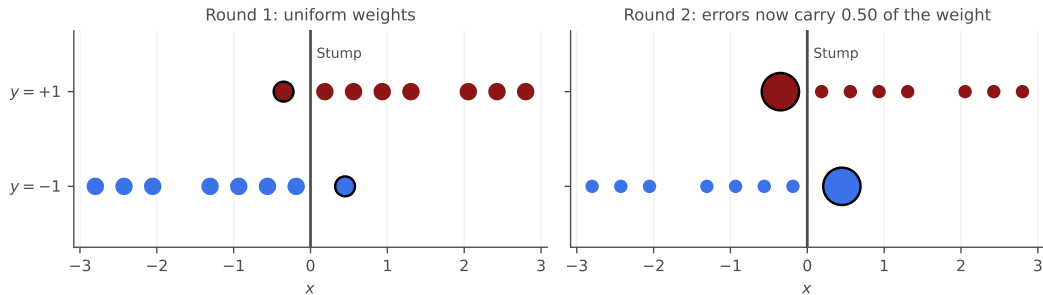
AdaBoost

Labels $y_i \in \{-1, +1\}$, weights initialized to $w_i = 1/N$. For $m = 1, \dots, M$:

1. Fit a weak classifier h_m to the data weighted by w_i .
2. Weighted error $\epsilon_m = \sum_i w_i 1\{y_i \neq h_m(\mathbf{x}_i)\}$.
3. Vote $\alpha_m = \frac{1}{2} \log \frac{1-\epsilon_m}{\epsilon_m}$.
4. Update $w_i \leftarrow w_i e^{-\alpha_m y_i h_m(\mathbf{x}_i)}$, then normalize.

$$F(\mathbf{x}) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(\mathbf{x}) \right).$$

AdaBoost weights



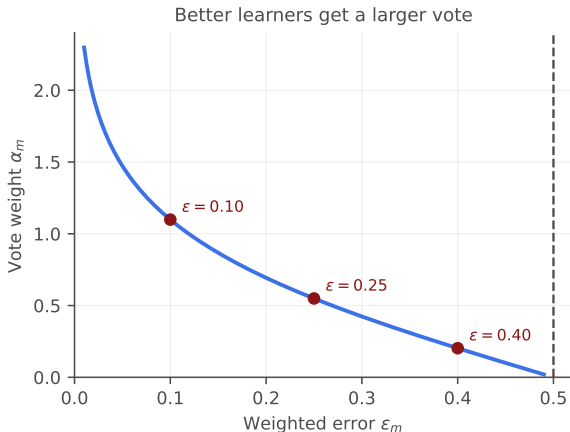
Black edge = misclassified by the stump. Here $\epsilon_1 = 1/8$ and $\alpha_1 = \frac{1}{2} \log 7 \approx 0.97$.

AdaBoost: classifier vote weight

The vote weight is

$$\alpha_m = \frac{1}{2} \log \frac{1 - \epsilon_m}{\epsilon_m}.$$

- Low weighted error $\Rightarrow$ large vote.
- Error near 0.5 $\Rightarrow$ almost no vote.
- Error above 0.5: flip the classifier.



AdaBoost estimates the log-odds

AdaBoost is forward stagewise
minimization of the **exponential loss**

$$L(y, F(x)) = e^{-y F(x)}.$$

AdaBoost estimates the log-odds

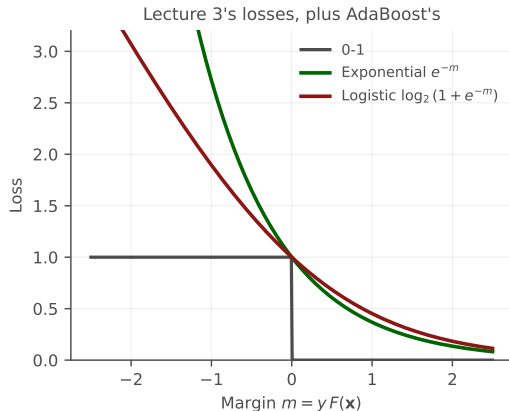
AdaBoost is forward stagewise minimization of the **exponential loss**

$$L(y, F(\mathbf{x})) = e^{-y F(\mathbf{x})}.$$

Its population minimizer is

$$F^*(\mathbf{x}) = \frac{1}{2} \log \frac{P(y = +1 \mid \mathbf{x})}{P(y = -1 \mid \mathbf{x})}.$$

Half the log-odds, the quantity that logistic regression models linearly.



Gradient boosting

Gradient boosting (Friedman, 2001) works with any differentiable loss L .

At stage m , compute the **pseudo-residuals**

$$r_i^{(m)} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}},$$

fit a small regression tree h_m to them, and update

$$F_m = F_{m-1} + \nu h_m.$$

Gradient boosting

Gradient boosting (Friedman, 2001) works with any differentiable loss L .

At stage m , compute the **pseudo-residuals**

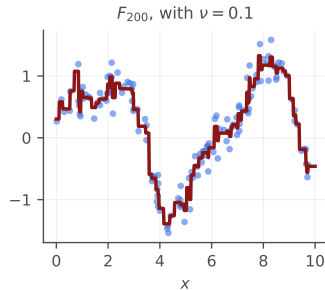
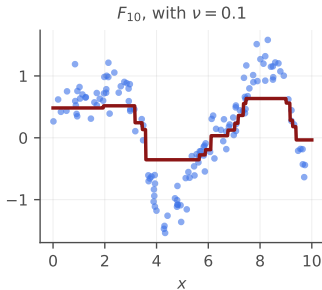
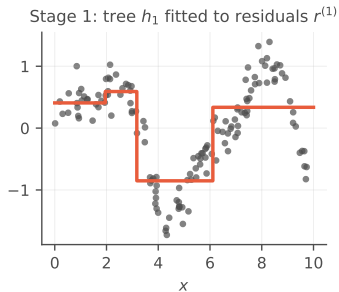
$$r_i^{(m)} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}},$$

fit a small regression tree h_m to them, and update

$$F_m = F_{m-1} + \nu h_m.$$

- Squared error: $r_i = y_i - F_{m-1}(\mathbf{x}_i)$, the ordinary residuals.
- Logistic loss, $y_i \in \{0, 1\}$: $r_i = y_i - \hat{p}_i$, **the gradient of lecture 3.**

Gradient boosting in action



Shrinkage and the number of stages

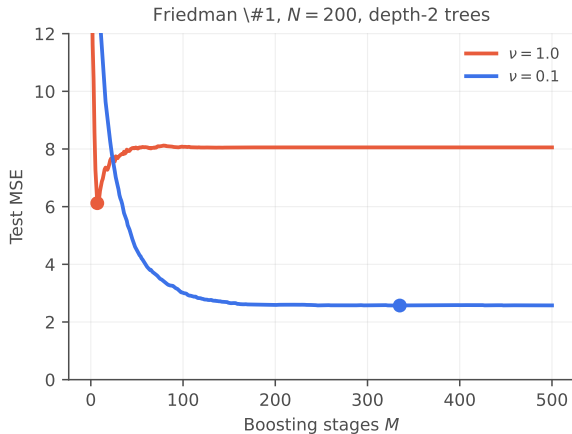
Boosting **can** overfit: the number of stages M is a complexity parameter.

A small learning rate ν

$$F_m = F_{m-1} + \nu h_m,$$

with $0 < \nu \leq 1$, needs more stages but reaches a lower test error.

Choose M by validation or early stopping.



Bagging vs boosting

Aspect	Bagging / random forest	Boosting
Training	parallel	sequential
Base trees	deep, high variance	shallow, weak learners
Main effect	variance reduction	bias reduction
Diversity	bootstrap / features	reweighting / residuals
More trees	never hurts	can overfit
Noise sensitivity	low	can be high

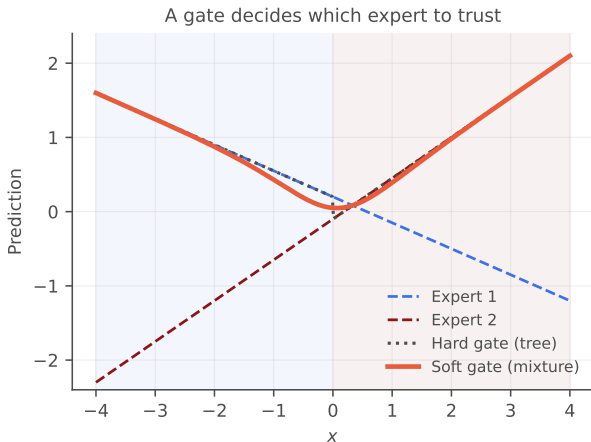
Both work because they exploit complementary errors.

Experts and partitions

Mixture of experts

A mixture of experts (Jacobs et al., 1991) combines specialized models $f_m(\mathbf{x})$ with input-dependent weights given by a gating function $g_m(\mathbf{x})$:

$$F(\mathbf{x}) = \sum_{m=1}^M g_m(\mathbf{x}) f_m(\mathbf{x}),$$
$$\sum_m g_m(\mathbf{x}) = 1.$$



Trees as hard mixtures

Recall the tree as a basis expansion:

$$T(\mathbf{x}) = \sum_{\ell=1}^J 1\{\mathbf{x} \in R_{\ell}\} c_{\ell}.$$

Each leaf is a constant expert c_{ℓ} ; the indicators are a hard gate.

Trees as hard mixtures

Recall the tree as a basis expansion:

$$T(\mathbf{x}) = \sum_{\ell=1}^J 1\{\mathbf{x} \in R_{\ell}\} c_{\ell}.$$

Each leaf is a constant expert c_{ℓ} ; the indicators are a hard gate.

A tree routes with deterministic, axis-aligned rules.

A mixture of experts makes the routing soft and learnable.

With fuzzy memberships as gates and linear experts, this is a TSK system (lecture 5).

In practice

Practical guidance

For tree ensembles:

- Start with a random forest as a robust baseline.
- Tune leaf size, depth, and features per split; use enough trees.
- Use gradient boosting when accuracy matters and validation is careful.
- Inspect calibration and class imbalance.
- Do not interpret feature importance as causality.

Trees handle non-linearities and interactions with little preprocessing.

Model calibration

A classifier is **calibrated** when predicted probabilities match observed frequencies.

If the model says

$$\hat{p}(y = 1 \mid \mathbf{x}) = 0.8$$

for many cases, then about 80% of those cases should be positive.

Accuracy and calibration are different questions.

Checking and fixing calibration

Diagnostics:

- Reliability diagram: predicted probability vs observed frequency.
- Brier score: mean squared error of the probabilities.
- Expected calibration error across probability bins.

Post-processing:

- **Platt scaling**: a logistic regression on the model's score.
- **Isotonic regression**: a monotone non-parametric map.

Calibrate with held-out or cross-validated predictions.

Feature importance: impurity reduction

Every split s reduces impurity by ΔQ_s , weighted by the fraction of samples reaching it. For feature j in a forest:

$$I_j = \frac{1}{B} \sum_{b=1}^B \sum_{s \in S_{b,j}} \frac{N_s}{N} \Delta Q_s,$$

where $S_{b,j}$ are the splits of tree b that use feature j .

A feature is important if it repeatedly makes nodes purer.

Also called mean decrease in impurity (MDI). This is `feature_importances_` in scikit-learn.

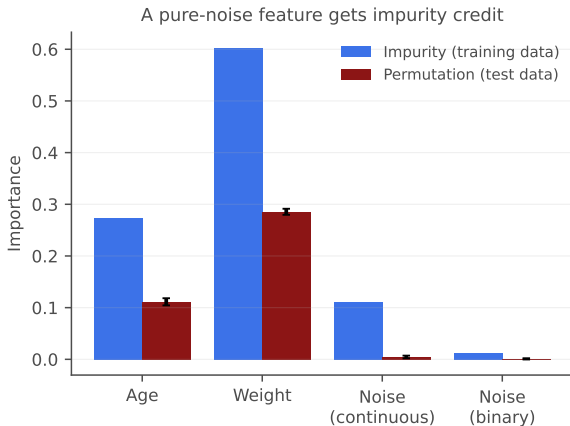
Feature importance: permutation

On held-out (or OOB) data:

1. Compute the baseline score.
2. Shuffle the column x_j .
3. Score again with the same model.

$$I_j = \text{score}(X, y) - \text{score}(X_{\pi(j)}, y).$$

If shuffling a feature hurts, the model was using it.



Missing values in tree ensembles

Missing values raise two different questions:

1. **Routing**: where does a tree send an example when x_j is missing?
2. **Statistics**: why is the value missing, and is that informative?

A random forest is not automatically immune to missing data.

The model or the pipeline must define a missing-value policy.

Common policies for missing values

Policy	Idea	When useful
Imputation	fill values before training	simple baseline
Missingness flag	add an <code>is_missing</code> indicator	missingness is predictive
Surrogate split	use a correlated feature	classic CART, <code>rpart</code>
Default direction	learn the path for missing values	boosting, recent <code>scikit-learn</code>

Always check what your library actually supports.

Missingness as signal

Trees suit incomplete data better than many linear models because decisions are local:

$$\text{At node } s : \begin{cases} x_j \leq t & \rightarrow \text{left} \\ x_j > t & \rightarrow \text{right} \\ x_j \text{ missing} & \rightarrow \text{learned path} \end{cases}$$

But missing data can still encode:

- Measurement failure.
- Selection bias.
- A clinically or operationally meaningful state.

Treat missingness as signal plus uncertainty.

Summary

- A tree is a linear model in a **learned basis** of indicators, grown greedily by impurity reduction.
- Classification trees reduce Gini or entropy; regression trees reduce squared error.
- **Cost-complexity pruning** is $\text{fit} + \alpha |T|$: the regularization of lecture 2.
- A single tree is interpretable but high-variance. Averaging has a floor $\rho \sigma^2$.
- **Bagging** averages deep trees; **random forests** also decorrelate them.
- **Boosting** adds small trees in sequence; AdaBoost estimates **half the log-odds**.
- **Gradient boosting** fits trees to $-\partial L / \partial F$; with logistic loss, to $y - \hat{p}$.
- Trees are hard mixtures of experts; soft gates lead to mixtures of experts and TSK.

References

- L. Breiman, J. Friedman, R. Olshen, C. Stone. *Classification and Regression Trees*. Wadsworth, 1984.
- L. Breiman. *Bagging predictors*. Machine Learning, 24(2):123–140, 1996.
- L. Breiman. *Random forests*. Machine Learning, 45(1):5–32, 2001.
- Y. Freund, R. Schapire. *A decision-theoretic generalization of on-line learning and an application to boosting*. J. Computer and System Sciences, 55(1):119–139, 1997.
- J. Friedman, T. Hastie, R. Tibshirani. *Additive logistic regression: a statistical view of boosting*. Annals of Statistics, 28(2):337–407, 2000.
- J. Friedman. *Greedy function approximation: a gradient boosting machine*. Annals of Statistics, 29(5):1189–1232, 2001.
- R. Jacobs, M. Jordan, S. Nowlan, G. Hinton. *Adaptive mixtures of local experts*. Neural Computation, 3(1):79–87, 1991.
- Y. Liu, X. Yao. *Ensemble learning via negative correlation*. Neural Networks, 12(10):1399–1404, 1999.
- T. Hastie, R. Tibshirani, J. Friedman. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.